Evaluating Token Orderings for LLM Pre-training

Wesley Kwiecinski[1]

[1]University of Illinois Chicago

August 20, 2026

1 Introduction

Pre-training for large language models involves massive and diverse data corpuses. A common corpus is the dolma[7] corpus, consisting of data across the web, stem papers, textbooks, and more. We also consider how each subcorpus is weighted in the final training set, and efforts have been made to produce a framework for determining how weights are updated in the pre-training process[1]. However, it's not just the diversity of data that's important, but also how that data is loaded and processed. Data is often randomized per epoch in order to improve generalization and learn the distribution of the data[5]. We want to understand if using certain token orderings improves training faster, either through a steeper loss curve or faster improvement in downstream tasks.

2 Methodology

2.1 Data Clustering

We start by doing clustering on a data corpus to determine good orderings of data. The corpus that we use is the olmo-mix[6] corpus. It consists of 6+ TB of data from the dclm, starcoder, pes2o, arxiv, open-web-math, algebraic-stack, and wiki datasets. The code for data processing is located on Github[4]. For exploration, we give a similar sample rate n to every subcorpus in the dataset to have a subset of the documents with similar ratios to the full corpus. Each file in the subcorpus is processed and each document in the file has a chance to be included in the final training set based on n. Once all documents in the final training set have been gathered, the first 2000 characters (about 300 words) of the text are passed to ALCF's LLM inference service. The LLM gives each document a set of attributes in a JSON format for later use. Gemma7b was used as the inference model on the Sambanova cluster (metis). For text attributes, we used: most likely categories, average difficulty, prior knowledge, vocab complexity, sentence quality, stem like, and code percentage. Also included are metadata attributes about the document such as number of tokens, subcorpus, and language.

Figure 1 shows the results for 20,000 documents from the corpus. Given similar sample rates for each subcorpus, there is a clear bias towards STEM subjects in the primary categories. The LLM is also only really confident up to 4 categories, as the probability spread for each subcategory becomes much larger than the previous. The attributes of each document show a clear correlation between STEM subjects and average difficulty of the document. Documents all seem to have good sentence quality, given that the corpus consists of screened and filtered data. There is a clear correlation between quality metrics and the difficulty of a document, mainly vocab complexity and prior knowledge.

Figures 2 and 3 show the clusters of documents according to topic and difficulty respectively. There is a large similarity between documents that contain STEM subjects. Notably, there are some primary document clusters that share the same primary category despite being very far apart from each other. "Medicine" is one such category. Most categories remain very close to each other, though sports and geography tend to be much farther away than the other subjects. Figure 3 shows that STEM subjects are often seen as more difficult. Given that there is such a large distance between the stem and less stem subjects, it may be good to order documents based on their difficulty and how close to a STEM topic they are. If each ordering is significant, we should see similar behavior between the two orderings. We'll also order documents based on how much code they contain, as an-

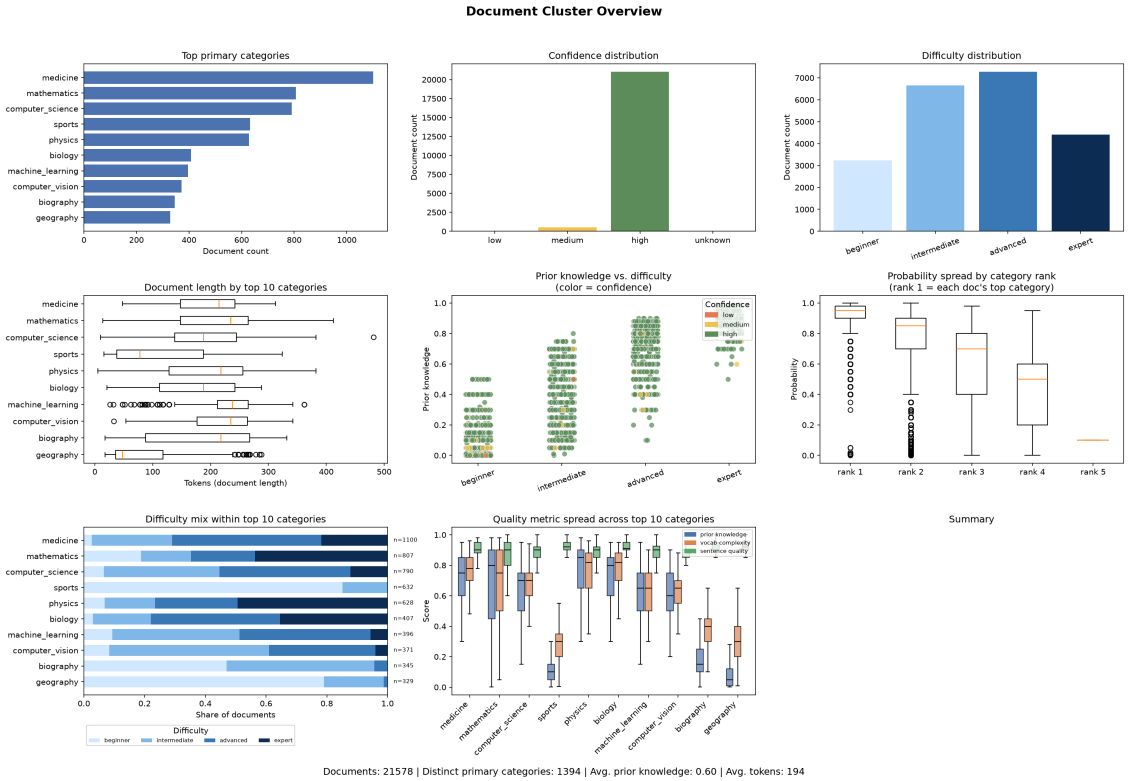


Figure 1: Cluster dashboard showing various metrics about the corpus subset.

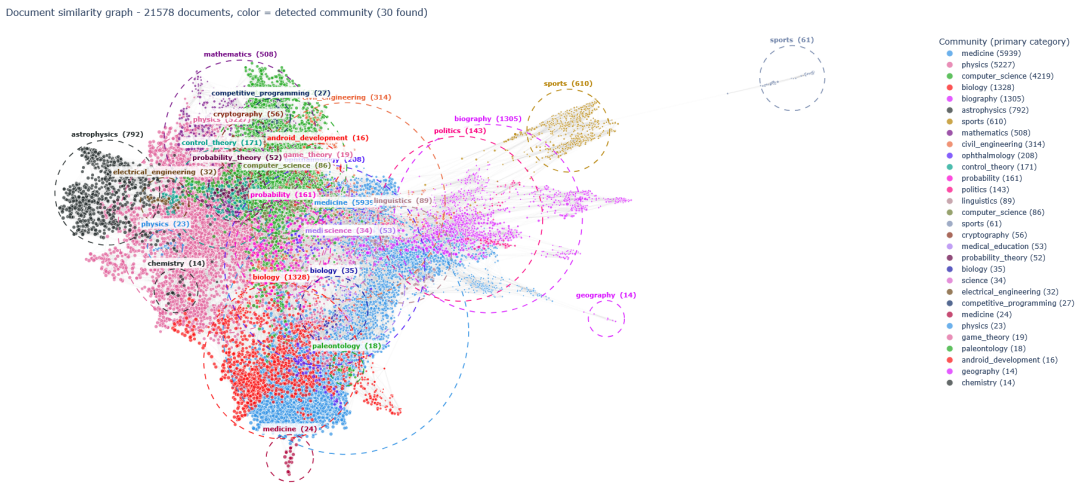


Figure 2: The clusters of documents by primary topic.

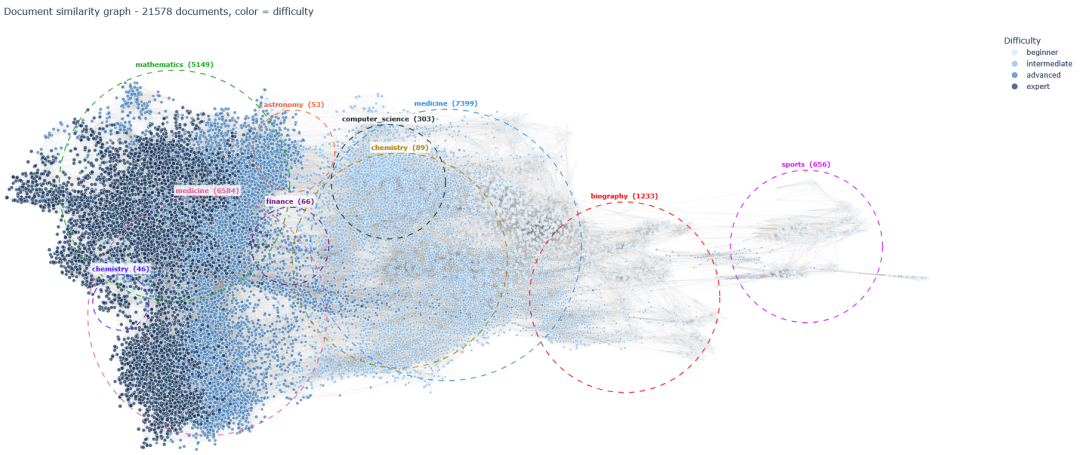


Figure 3: The clusters of documents by difficulty.

other slight variation that might show different behavior similar to difficulty and STEM. We'll also use a random ordering based on a seed as a control group. We also use the reversed orderings of each token ordering to see if training becomes worse.

2.2 Training Pipeline

For setting up the training pipeline, we use a fork of torchtitan that contains experiments set up with ezpz. The goal is to train a smaller AuroraGPT model as a proxy for larger models. OLMIX shows that smaller model swarms can be a good base for larger models, notably a model with 15M parameters has good rank correlation with a model with 1B parameters[1]. So, we use a 70M parameter model with about 1B tokens, a global batch size of 8, (local batch 2, 4 gpus), and 3 epochs (about 15000 steps per epoch). Shuffling is disabled by using a streaming mode through torchtitan's HuggingFaceTextDataLoader to ensure that the token ordering is kept across each epoch. We set up 20 checkpoints, with a checkpoint being created every 1000 steps. The full pipeline is as follows: 1) Initial data pre-processing, 2) Cluster and order data according to some arbitrary metric, 3) train each agpt configuration with the specified ordering of data, 4) evaluate each generated checkpoint and save the results. For training, we processed about 400,000 documents, close to 1B total tokens according to the Llama2[8] tokenizer. This tokenizer was chosen over the default of Gemma7B for the AuroraGPT due to its smaller vocab size, about 32,000 compared to Gemma7B's 256,000, to reduce the overall size of the non-embedded parameters of the final model.

The orderings used in training for this experiment are: "none", "difficulty", "difficulty-reverse", "stem", "stem-reverse", "code", and "code-reverse". The "-reverse" suffix performs the requested ordering, but the final order is flipped. The "none" ordering serves a baseline to compare with the other orderings. The data is gathered into a single json file and randomized according to a seed. The "difficulty" and "difficulty-reverse" orderings cluster the data into 4 clusters, with weights biased towards the "vocab complexity", "sentence quality", "average difficulty", and "prior knowledge". Clusters are ordered by their average highest value, and the documents are ordered based on their proximity to the cluster. Documents are ordered from least difficult to most. The "stem" and "stem-reverse" orderings order all documents by the "stem_like" value determined by the LLM inference. The same is done for the "code" and "code-reverse" orderings.

2.3 Evaluation Pipeline

For the evaluation pipeline, we use a small set of tasks from lm_eval to evaluate how the learning is affected over time. We use a set of common LLM benchmarks for determining prediction, knowledge, and mathematical performance. The full set of benchmarks include: arc_easy, openbookqa, lambada_openai, truthfulqa, wikitext, and gsm8k. Each evaluation suite is ran per-checkpoint to see the changes in accuracy over time. Each task is ran with 5 shots. Once a task suite has been finished, the json containing the summaries result of each task is saved and used for processing. Each json is then sent through a visualization step to create the final training dashboard.

3 Results

The final training dashboard for each ordering is seen in figure 4. The left column shows the loss and gradient normal curves for each order. The "difficulty" and "difficult-reverse" orderings both perform similarly, but they have significantly steeper initial loss and gradient normal curves compared to the other orderings. Oddly, the two orderings seem to have the same loss curves despite being inverses of each other. All other orderings seem to perform similarly to each other, but we see some inverse behavior between the "stem" and "stem-reverse" orderings, where loss seems to increase or decrease at the end of an epoch. Despite this, they still stay similar to the loss curves of the baseline and code orderings. The gradient normal curves stay similar between all orderings, with the "difficulty" and "difficulty-reverse" orderings showing a slow drop off.

The right column of figure 4 shows the results of each test suite for each ordering. The top plot shows the results of the arc_easy and openbookqa tasks. Each ordering performs similarly to each other, but each difficulty ordering had noticeably worse performance on arc_easy. However, there's no difference in openbookqa. This is likely due to openbookqa being a general knowledge evaluation task based on openbook exams. Each ordering might not change the overall accuracy due to the lack of tokens used in training. We also likely see the same issue in the truthfulqa and wikitext benchmarks. The results from the wikitext benchmark are interesting, as this is where the first divergence between the "difficulty" and "difficult-reverse" orderings appear. Ordering the documents from hard to easy performs slightly worse than easy to hard in terms of bits per byte. All token orderings seem to perform equally on gsm8k, though the "code" ordering (smallest to largest code ratio).

4 Conclusion

4.1 Summary

This pilot experiment testing different token orderings for 1B tokens on a 70M AuroraGPT model in order to determine how different orderings affect training and downstream tasks. The corpus data was first processed and attributes were added through the use of an LLM inference service. Then, the data was ordered according to arbitrary metrics determined by clusters generated using the attributes provided by the LLM. Once orderings were created, a model was generated and tested on a task suite using lm eval for each checkpoint in each ordering. We found that certain orderings can have steeper loss and gradient normal curves, but may ultimately perform worse on downstream tasks. While smaller models can provide similar results to larger models with good rank correlation[1], it's hard to say if we would see similar loss curves on larger AGPT models.

4.2 Limitations and Future Work

There were a few different limitations to this study that likely impacted the final results. The first major one was that the number of tokens that was used was a bit under the necessary amount for the Chinchilla optimal ratio[2]. Ideally, we would train a significantly larger model with a larger number of tokens. As it stands, the current model configuration that is used is close to GPT-2, which is certainly not state-of-the-art anymore. A large next step would be to process a significantly larger portion of the corpus. There were also a small subset of documents in the final pre-training set that were not english documents, which likely impacted the final results of each evaluation task.

The checkpoints for each task suite evaluation likely don't tell us as much as they could, due to each checkpoint appearing after the first epoch. Given that the loss curves drop more quickly within the first epoch for all orderings, using a checkpoint spread over larger steps would likely reveal more information about whether or not training improves faster for each difficulty ordering. It may also show that the difficulty orderings have worse performance after 3 epochs due to over-tuning or insufficient data.

Limitations on how data was being processed also had a large impact on how many tokens were in the final training set. Because an LLM inference service was used to determine text attributes, we were forced to deal with rate limits any time we wanted to gather attributes about each document. This impacted how many documents could actually be processed within a certain time frame. Using other, classical algorithms for determining some text attributes, like the Flesch-Kincaid[3] score for readability, could speed up processing. However, classical algorithms often suffer from being too simplistic to gather meaningful results from, as well as being unable to gather semantic meaning from texts. This is one of the largest limiting factors of using data ordering and clustering for pre-training. Also, because clustering doesn't have an inherent ordering or meaning without a human-in-the-loop, it's hard to automatically determine

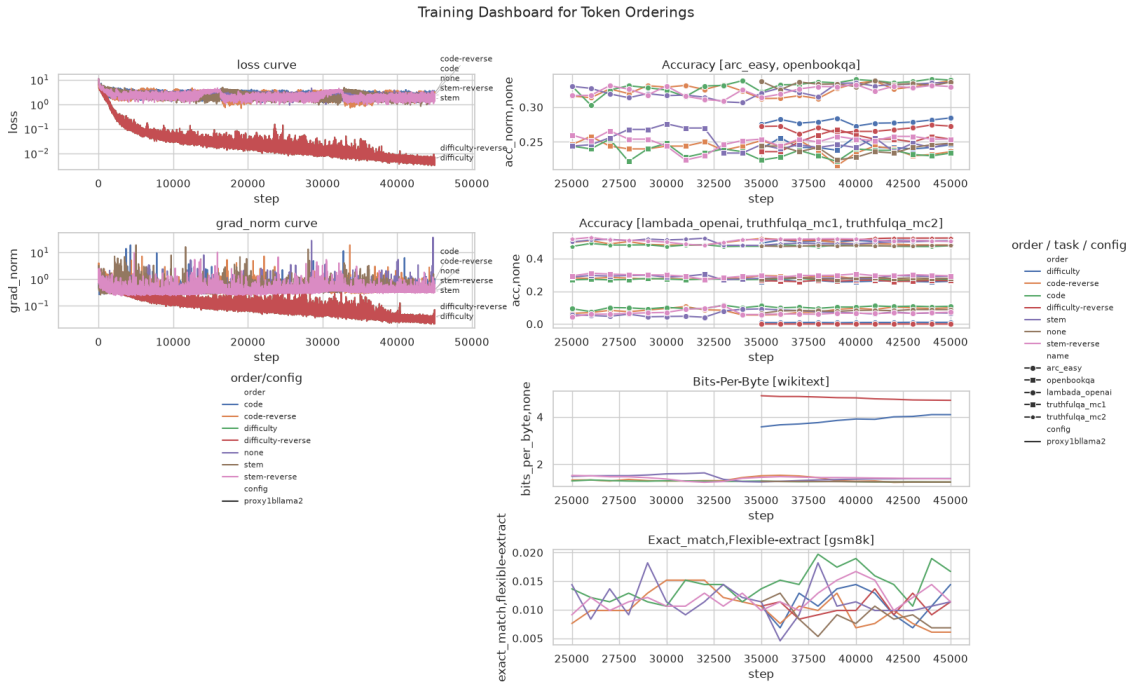


Figure 4: The training dashboard for model training with each ordering.

meaningful orderings that could impact training. It seems that the overhead cost of the time spent ordering data beforehand may not justify any learning or downstream task improvements gained from doing so.

References

[1] Mayee F Chen, Tyler Murray, David Heineman, Matt Jordan, Hannaneh Hajishirzi, Christopher Ré, Luca Soldaini, and Kyle Lo. Olmix: A framework for data mixing throughout lm development. *arXiv preprint arXiv:2602.12237*, 2026.

[2] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, et al. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*, 2022.

[3] J Peter Kincaid, Robert P Fishburne Jr, Richard L Rogers, and Brad S Chissom. Derivation of new readability formulas (automated readability index, fog count and flesch reading ease formula) for navy enlisted personnel. 1975.

[4] Wesley Kwiecinski. Corpus Exploration. https://github.com/WesKw/CorpusExploration.

[5] Truong Thao Nguyen, François Trahay, Jens Domke, Aleksandr Drozd, Emil Vatai, Jianwei Liao, Mohamed Wahib, and Balazs Gerofi. Why globally re-shuffle? revisiting data shuffling in large scale deep learning. In *2022 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pages 1085–1096. IEEE, 2022.

[6] Team Olmo, Allyson Ettinger, Amanda Bertsch, Bailey Kuehl, David Graham, David Heineman, Dirk Groeneveld, Faeze Brahman, Finbarr Timbers, Hamish Ivison, et al. Olmo 3. *arXiv preprint arXiv:2512.13961*, 2025.

[7] Luca Soldaini, Rodney Kinney, Akshita Bhagia, Dustin Schwenk, David Atkinson, Russell Authur, Ben Bogin, Khyathi Chandu, Jennifer Dumas, Yanai Elazar, et al. Dolma: An open corpus of three trillion tokens for language model pretraining research. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 15725–15788, 2024.

[8] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.